

Semana 6 — Resource customizado (ItemData) e Enums

Semana 6

Resource customizado (ItemData) e Enums

Disciplina: Tendências de Motores de Jogos (IN46A)

Curso: Tecnologia em Jogos Digitais — IFMS Campus Dourados

Unidade II — Construir Sistemas (Semanas 4–7)

Projeto: Vertical Slice *O Templo Esquecido*

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Objetivos da Semana

- Compreender Resource customizado como estrutura de dados desacoplada da lógica de gameplay
- Compreender Enums como organizadores de dados tipados dentro de um Resource
- Modelar um conjunto de itens coletáveis do Vertical Slice usando `ItemData` + Enum

Semana 6 — Resource customizado (ItemData) e Enums

ENCONTRO 1

O Resource ItemData

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Agenda do Encontro 1

- Revisão do Encontro 2 da Semana 5 (objeto interativo com contrato `Interactable` + `Signal`) (15 min)
- Introdução: o problema de misturar dado de design e lógica de gameplay (20 min)
- Demonstração: criação do Resource `ItemData` e de instâncias `.tres` (35 min)
- Laboratório: cada estudante cria seu próprio `ItemData` e ao menos duas instâncias de itens (45 min)
- Desafio: adicionar um campo próprio ao `ItemData` não demonstrado em aula (15 min)
- Feedback e fechamento (5 min)



Se um item também precisasse aparecer em uma prateleira de loja ou em um slot de inventário, os mesmos dados seriam duplicados em cada lugar?

Pense em nome, ícone e valor declarados soltos dentro do script de um baú.

O Problema: Dado de Design Misturado à Lógica

- Toda engine com conteúdo além do mínimo precisa resolver o mesmo problema: como permitir que quem projeta o conteúdo trabalhe sem editar código a cada novo item
- Se cada item exigisse uma classe própria ou valores hardcoded no script do objeto, adicionar um item novo significaria mexer em lógica, não em dado
- É o mesmo princípio do contrato `Interactable` da Semana 5, em outra escala: ali desacoplou-se comportamento; aqui desacopla-se dado

O Resource Customizado `ItemData`

- Estrutura de dados serializável, independente de qualquer Scene ou Node específico
- Definido por um script que estende `Resource` e declara `class_name`
- Campos `@export` ficam visíveis e editáveis no Inspector, sem estar preso a nenhuma cena
- O mesmo `ItemData` pode ser reutilizado por um baú, uma loja ou um slot de inventário

Dado de Design como Asset no Godot × Unity

Godot

- Resource: classe-base também usada por texturas, cenas e scripts
- Customizar um Resource estende um mecanismo já onipresente no motor

Unity

- `ScriptableObject` : classe-base separada, criada especificamente para esse padrão
- Exige `[CreateAssetMenu]` para aparecer no menu de criação de assets

Demonstração — Criando o `ItemData`

O que será construído:

- Classe `ItemData`, estendendo `Resource`, com campos `nome`, `icone`, `valor`, `descricao`
- Duas instâncias `.tres` distintas (`item_moeda.tres`, `item_chave.tres`)
- Preenchimento dos campos pelo Inspector, sem nenhuma linha de código adicional

Por quê: primeiro dado de design desacoplado do Vertical Slice, base de todo item até a ampliação do Inventário na Semana 10.

Imagem sugerida

Objetivo: mostrar o script de um Resource customizado no editor de script do Godot, com `class_name ItemData` e campos `@export` declarados.

Enquadramento: captura de tela do Script Editor com o arquivo `item_data.gd` aberto.

Elementos presentes: a linha `extends Resource`, a linha `class_name ItemData`, ao menos dois campos `@export`.

Destaque visual: a palavra-chave `extends Resource`, que diferencia este script de um script de Node comum.

Legenda sugerida: "Script ItemData estendendo Resource, com campos exportados para nome, ícone e valor."

Arquitetura — Classe e Instância

- `item_data.gd` (classe): define a estrutura — quais campos todo item possui
- `item_moeda.tres`, `item_chave.tres` (instâncias): dados concretos preenchidos a partir da classe
- Nenhuma instância altera a estrutura — apenas preenche valores próprios
- Convenção do projeto: classe em PascalCase, instância em snake_case descrevendo o item

Laboratório — ItemData e Instâncias Próprias

Cada estudante replica, no próprio projeto:

1. Pasta `resources/items/` criada, conforme `PROJECT_ARCHITECTURE.md`
2. Classe `ItemData` em `scripts/resources/item_data.gd`, estendendo `Resource`
3. Campos exportados: `nome`, `icone`, `valor`, `descricao`
4. Pelo menos duas instâncias `.tres` distintas, com valores próprios preenchidos no Inspector

Boas Práticas — ItemData

- Dar valores padrão aos campos exportados, evitando instâncias com campos vazios por esquecimento
- Nomear cada instância `.tres` descrevendo o item concreto, nunca de forma genérica
- Manter todas as instâncias de item na mesma pasta (`resources/items/`)
- Reservar ao `ItemData` apenas dado — a reação ao item pertence à Scene que o consome



Desafio — Encontro 1

Adicione ao próprio `ItemData` um campo não demonstrado em aula — por exemplo, peso, raridade ou uma segunda descrição curta para tooltip.

OBJETIVOS

Mantenha a estrutura de Resource como único ponto de entrada dos dados — nenhuma lógica de gameplay entra no `ItemData`.

Fechamento — Encontro 1

- Classe `ItemData` (Resource customizado) definida, com `nome`, `icone`, `valor`, `descricao`
- Pelo menos duas instâncias `.tres` de itens distintos, preenchidas no Inspector
- Campo adicional do desafio incluído e testado
- Próximo passo: Enum de categoria, no Encontro 2

Semana 6 — Resource customizado (ItemData) e Enums

ENCONTRO 2

Enums e o Conjunto de Itens do Grupo

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Agenda do Encontro 2

- Revisão do Encontro 1 (`ItemData` criado e instâncias `.tres` de itens) (10 min)
- Introdução: Enums como organizadores de um conjunto fechado de valores (20 min)
- Demonstração: declaração de um Enum de categoria e aplicação ao `ItemData` (35 min)
- Laboratório e Desafio: cada grupo modela seu próprio conjunto de itens coletáveis usando `ItemData` + Enum (50 min)
- Checkpoint de progresso do Módulo 2 — apresentação rápida dos conjuntos de itens (20 min)



Se dez itens diferentes puderem pertencer a apenas três categorias, faz sentido cada um digitar a categoria como texto livre?

Pense no que acontece se um item receber "moeda", outro "Moeda" e outro "moedas".

Enums como Conjunto Fechado de Valores

- O `ItemData` resolve "onde vive o dado"; o Enum resolve "como impedir valores inconsistentes em um campo"
- Presente em praticamente toda linguagem de programação — o mesmo problema aparece em qualquer domínio
- No Godot, um Enum é declarado dentro do script do Resource
- Um campo `@export` do tipo do Enum gera automaticamente um menu suspenso no Inspector

Campo Tipado no Godot × Unity

Godot

- `enum Categoria { MOEDA, RECURSO, CHAVE }` dentro do próprio script
- Valores tratados como inteiros nomeados, sem arquivo separado

Unity

- `enum Categoria` em C#, aplicado a um campo `[SerializeField]`
- C# permite declarar o `enum` fora da classe que o usa

Demonstração — Enum Categoria no ItemData

O que será construído:

- `enum Categoria { MOEDA, RECURSO, CHAVE }` declarado em `item_data.gd`
- Campo `@export var categoria: Categoria` adicionado à classe
- Menu suspenso resultante aplicado às instâncias `item_moeda.tres` e `item_chave.tres`

Por quê: elimina a possibilidade de um valor de categoria fora do conjunto esperado, mantendo consistência entre todas as instâncias do projeto.

Imagem sugerida

*Objetivo: mostrar o campo **categoria** do **ItemData** no Inspector, exibido como menu suspenso gerado a partir do Enum.*

*Enquadramento: captura de tela do Inspector com uma instância **.tres** de item selecionada.*

*Elementos presentes: o campo **categoria**, o menu suspenso aberto mostrando as opções do Enum (MOEDA, RECURSO, CHAVE).*

Destaque visual: o menu suspenso, contrastando com um campo de texto livre.

Legenda sugerida: "Campo categoria do ItemData exibido como menu suspenso gerado pelo Enum."

Arquitetura — ItemData + Enum

- `ItemData` (Resource): define a estrutura de campos, incluindo `categoria`
- `Categoria` (Enum): restringe `categoria` a um conjunto fechado e nomeado
- Cada instância `.tres` escolhe um valor do Enum pelo menu suspenso do Inspector
- Qualquer grupo pode ajustar os valores do Enum ao próprio tema, sem alterar a estrutura da classe

Laboratório e Desafio — Conjunto de Itens do Grupo

Cada grupo modela seu próprio conjunto de itens coletáveis:

- Ajusta o Enum `Categoria` ao tema do próprio Vertical Slice, se necessário (três a cinco categorias)
- Cria pelo menos três instâncias `.tres` distintas, com todos os campos preenchidos, incluindo `categoria`
- Liberdade de categorias e atributos — mesma estrutura de `ItemData` para todos os grupos

Boas Práticas — Enum **Categoria**

- Nomear o Enum em PascalCase e seus valores em maiúsculas, conforme a convenção do projeto
- Manter o Enum pequeno (três a cinco categorias) — um Enum extenso geralmente indica que o conceito deveria ser outro campo
- Definir um valor padrão coerente no campo exportado
- Salvar cada instância `.tres` explicitamente após alterar `categoria` no Inspector



Desafio — Encontro 2

Cada grupo modela seu próprio conjunto de itens coletáveis (baús, moedas, recursos ou equivalente) usando `ItemData` + Enum, com liberdade de categorias e atributos.

OBJETIVOS

Entrega: Checkpoint de progresso do Módulo 2 — apresentação rápida do conjunto de itens de cada grupo.

Checkpoint — Progresso do Módulo 2

Ao final da semana, cada grupo possui:

- Classe `ItemData` (Resource customizado) com campos `nome`, `icone`, `valor`, `descricao`, `categoria`
- Enum `Categoria` com três a cinco valores, ajustado ao tema do próprio Vertical Slice
- Conjunto de três ou mais instâncias `.tres` de itens coletáveis, coerentes entre si

Fechamento — Encontro 2

- Enum `Categoria` declarado, aplicado ao `ItemData` e refletido no menu suspenso do Inspector
- Conjunto próprio de itens coletáveis por grupo, coerente com o tema do Vertical Slice
- Checkpoint de progresso do Módulo 2 apresentado
- Módulo 2 avança com dado e lógica desacoplados, sustentando todo item futuro

Resultado Esperado da Semana

- Classe `ItemData` (Resource customizado) com pelo menos um Enum de categoria
- Conjunto próprio de itens coletáveis por grupo, coerente com o tema do Vertical Slice
- Turma relaciona `ItemData` e Enum ao equivalente da Unity (`ScriptableObject` e `enum` de C#)
- Checkpoint de progresso do Módulo 2 concluído

Checklist da Semana

- [] Classe `ItemData` criada, estendendo `Resource`, com `class_name` definido
- [] Pelo menos quatro campos exportados (`nome`, `icone`, `valor`, `descricao`) visíveis no Inspector
- [] Enum `Categoria` declarado, com três a cinco valores em PascalCase/maiúsculas
- [] Campo `categoria` exportado, exibido como menu suspenso no Inspector
- [] Pelo menos três instâncias `.tres` de itens distintos, com todos os campos preenchidos
- [] Checkpoint de progresso do Módulo 2 apresentado com sucesso

Próximos Passos — Semana 7

O par `ItemData` + Enum construído nesta semana é pré-requisito direto da Semana 7, que introduz `SaveData` (Resource) + `FileAccess` para serializar e recuperar o estado de progresso do jogador, incluindo os itens coletados. A Semana 7 também integra, em um único fluxo jogável, todos os desafios do Módulo 2, encerrando a Unidade II com Code Review e Playtest coletivo.

Leitura recomendada: Godot Docs — Resources, GDScript (Enums); Unity Manual (consulta comparativa) — ScriptableObject.